

Quant Guild Recruitment

Trading Bot Competition: Problem Statement

1 Overview

You will program a trading bot that competes against other bots in a multi-round auction. Each round your bot is given a private value and must decide how much to bid. The objective is to maximise profit over the full game — across changing market conditions and changing bankrolls.

The competition runs over several **auction variations**, which differ only in how payoffs are assigned after the bids come in. Variations 1 and 2 are released now; further variations will be released after the first mock auction. Everything else in this document — the game mechanics, the scoring, the tournament structure — applies to every variation.

Please read this document in full. Post any questions on the guild WhatsApp group and we will respond as soon as we can.

2 Timeline

All times IST, dates in DD/MM.

When	What
Wed 09/09, 10:00 pm	Orientation. Variations 1 and 2 released with starter code.
Sat 12/09, 12:00 pm	Deadline for mock submissions of Variations 1 and 2 (optional, strongly recommended).
Sat 12/09, evening	Mock Auction 1 run; results shared. Two further variations released.
Mon 14/09, EOD	Deadline for mock submissions of the new variations. Mock Auction 2 run the same night; results shared.
Wed 16/09, 11:59 pm	Final submission deadline , all variations.

The mocks exist so you can watch your bot fail against a real field before it counts. Your final bot need not resemble your mock bot.

3 Game Mechanics

3.1 Structure

- **Players:** $n = 20$ bots per auction group.
- **Rounds:** $t = 2000$, split into **4 blocks of 500 rounds**.

- Each round, every active player i independently draws a value x_i from that block's distribution, submits a bid, and receives a payoff determined by the variation in play.
- Capital updates as new capital = old capital + payoff.

3.2 How a block is initialised

At the start of every block, the market is re-drawn from scratch. For block b :

1. Draw a **minimum** m_b uniformly from $\{100, 110, 120, \dots, 1000\}$ (multiples of 10).
2. Draw a **range** R_b uniformly from $\{100, 110, 120, \dots, 1000\}$ (multiples of 10).
3. For every round in that block, values are drawn as $x_i \sim \mathcal{U}[m_b, m_b + R_b]$, independently for each player and each round.

You are told none of this. Not m_b , not R_b , not the block index, and not when a block boundary has occurred. Detecting the regime change and estimating the distribution from what you observe is a core part of the problem.

3.3 Starting capital

Capital is also re-drawn at every block boundary, **independently for each player**. For player i in block b :

$$k_i \sim \mathcal{U}[0.5, 2.5], \quad C_i^{(b,0)} = k_i \cdot R_b + m_b.$$

So within the same block some bots start rich and some start poor, and nobody is told anyone else's capital. Your maximum legal bid is always your current capital.

3.4 Capital does not carry over

The capital you finish a block with is banked as score, not as money. It is normalised and converted into leaderboard points (Section 6); the next block then starts from a freshly drawn capital.

The reason: varying the starting capital is a test of whether your bot adjusts its aggression to its bankroll — $0.5\times$ the range is a very different situation from $2.5\times$. If capital carried over, that test would break. A bot that drew a large bankroll early and compounded it could stay aggressive for the rest of the game purely because it is rich, so the later blocks would be measuring early luck instead of strategy. Resetting makes each block a clean, independent test, and the leaderboard aggregates four of them per iteration.

A bot that goes bankrupt is out for the remainder of that block only. It returns at the next block boundary with a fresh capital draw.

3.5 Bids

- Bids may be fractional and must lie in $[0, \text{current capital}]$.
- An illegal bid (greater than capital, negative, NaN, or not returned in time) is automatically set to 0 for that round.
- If multiple players tie for the highest bid, all of them win and all of them receive the winner's payoff.

4 Information available to your bot

At the start of round t , your bot receives exactly the following:

- your own value x_i for this round;
- your current capital c_i (equivalently, your maximum legal bid);
- n_t , the **number of active players** this round (bots with non-zero capital);
- the round index t (1–2000);
- the **highest and second-highest bids** of the previous round;
- your own bid, rank and payoff in the previous round;
- **Variation 2 only:** the realised X of the previous round.

Notes:

- You never learn m_b , R_b , the block index, or any other player's value, capital or identity.
- n_t matters because in Variation 2 the payoff depends on the maximum value across the active players. As bots go bankrupt, n_t falls and that maximum falls with it in expectation.
- Your bot may keep internal state across rounds (history buffers, estimators, and so on). State persists for the whole 2000 rounds — it is *not* cleared at block boundaries, so you can use it to detect the change.

5 Auction Variations

Let b_1 be the highest bid in a round, and let $X = \max_j x_j$ over the players active that round.

1. **Variation 1 — private value, first price.**

The winner receives Payoff = $x_i - b_1$, where x_i is the *winner's own* value. Everyone else receives zero.

2. **Variation 2 — common value, first price.**

The winner receives Payoff = $X - b_1$, where X is the maximum value amongst all active players that round. Everyone else receives zero.

Note the difference: in Variation 1 you are bidding on something only you can see. In Variation 2 everyone is bidding on the same prize, and winning it is evidence that you overestimated it.

6 Scoring

Raw profit is not comparable across blocks — a block with $R_b = 1000$ produces payoffs an order of magnitude larger than one with $R_b = 100$, and starting capitals differ between players inside the same block. Profit is therefore normalised in two steps.

Step 1 — scale. For player i in block b ,

$$\pi_i^{(b)} = \frac{C_i^{(b,\text{end})} - C_i^{(b,0)}}{R_b},$$

i.e. profit measured in units of that block's range. A bankrupt bot has $C^{(b,\text{end})} = 0$, giving $\pi_i^{(b)} = -C_i^{(b,0)}/R_b$.

Step 2 — standardise within the group. Let μ_b, σ_b be the mean and standard deviation of $\pi^{(b)}$ over the 20 players in your group for that block. Then

$$z_i^{(b)} = \text{clip}\left(\frac{\pi_i^{(b)} - \mu_b}{\sigma_b}, -3, 3\right), \quad P_i^{(b)} = 50 + 15 z_i^{(b)} \in [5, 95].$$

Clipping at ± 3 prevents one freak block from deciding the competition.

$$\text{Iteration score} = \sum_{b=1}^4 P_i^{(b)}, \quad \text{Total} = \sum_{\text{iterations}} \text{Iteration score}.$$

Alongside the score we record and publish, per bot: mean normalised profit $\bar{\pi}$, survival rate, worst-block π , spread of π across blocks, and performance broken down by starting-capital decile.

7 Tournament Structure

A single 20-bot arrangement is far too noisy to rank on — who you are grouped with matters nearly as much as how you play. Each variation is therefore run over **5 iterations** with different random seeds. Seeds are published afterwards so results are reproducible.

- **Iterations 1 and 2 — random.** Bots are shuffled into groups of 20 under two independent seeds. Full 2000 rounds, 4 blocks each.
- **Iteration 3 — strength-balanced.** Cumulative points from iterations 1–2 are used to *balance* the groups, not to segregate them: bots are sorted by points and dealt into groups in a snake pattern, so every group has similar average strength. This sharply reduces the variance of the standardised score while keeping cross-group comparison fair.
- **Iterations 4 and 5 — finals.** The top 20 bots after iteration 3 play two head-to-head iterations under fresh seeds. Final ranking is the sum of these two, tie-broken by the qualification total.

That is 12 scored blocks in qualification and 8 in the finals — enough for a bot’s mean score to stabilise well below the gaps we care about.

The **mock auctions** use the same machinery at reduced size: **2 randomly seeded iterations**, with per-block and per-iteration statistics published so you can see where your bot broke.

8 Sample Round

Three bots, capital 500 each. Suppose the hidden block has $m_b = 200$, $R_b = 300$, so values fall in $[200, 500]$.

- Bot 1 — always bids 15 more than its own value.
- Bot 2 — always bids 5 more than its own value.
- Bot 3 — always bids half its own value.

The values drawn are 230, 250, 260, so $X = 260$. The bids are 245, 255, 130. **Bot 2 wins.**

	Variation 1	Variation 2
Bot 2’s payoff	$250 - 255 = -5$	$260 - 255 = +5$
Capital after	$500 / \mathbf{495} / 500$	$500 / \mathbf{505} / 500$

Bots 1 and 3 receive zero payoff in both cases.

9 Deliverables

Submit your bot code (Python files following the provided template) together with a **short report** explaining your strategy.

- One file per variation. If your Roll No. is OB24C420, submit `OB24C420_1.py` for Variation 1 and `OB24C420_2.py` for Variation 2. Later variations follow the same pattern.
- You may submit for only some variations.
- Put all files in a Google Drive folder called “Submissions”, give it view access, and submit the link here: [Submission Form](#). The same form and folder are used for the mocks — just update the contents before each deadline.

Your report should cover, for each variation: the core idea, how you estimate the hidden distribution, how your bid depends on your starting capital, how you detect and react to a block change, and which libraries you used.

Final deadline: Wednesday 16th September 2026, 11:59 pm.

10 Rules and Constraints

- Modify only `Template.py`. The other files are provided so you can see how the auction is simulated; see `README.md` for how to run it. Sample bots are included.
- Keep your bot under **1 second per round** and under **100 MB** of memory. Bots that exceed either may be removed from the auction.
- Your bot must not read or write files, spawn processes, access the network, or inspect the simulator’s internal state. Doing so is disqualification.
- Any Python library is allowed. List the ones you use in your report.
- All submissions are first run against our sample bots to check that they do not crash, do not go bankrupt immediately, and behave sensibly. Bots that pass enter the tournament.

11 What We Are Looking For

- **The report and the reasoning behind your strategy** — this is the primary metric.
- Net profit alone is not a metric. It is read alongside how the strategy adapts.
- **Robustness to starting capital:** does aggression genuinely respond to bankroll, or is it fixed?
- **Robustness across blocks:** consistency of π across the four regimes, and how fast the bot re-adapts after a boundary.
- **Survival:** bankruptcy is heavily penalised — it forfeits the rest of the block.

12 Resources

- Investopedia — Random Variables
- W3Schools — Python Introduction
- W3Schools — NumPy Introduction



Feel free to make extensive use of AI — we are testing the logic behind your strategy, not your coding skills. That said, please indent properly and comment where it helps us read your code.

ALL THE BEST!